MOMENTUS Codebase Blueprint

Prepared: 6 November 2026, Asia/Manila **Purpose:** A code-oriented map of the current application, its trust boundaries, data model, principal flows, and delivery state. **Baseline:** This blueprint describes the repository implementation. It does not replace or modify the unchanged product concept in `MOMENTUS_Product_Specification.pdf`. User-approved decisions and acceptance evidence remain authoritative in `DECISIONS.md` and `ACCEPTANCE.md`.

1. Executive view

MOMENTUS is a Next.js 16 App Router application organized as a modular monolith. It serves organizers, guests, and staff for private events. Supabase provides Auth and PostgreSQL with row-level security (RLS) and SQL RPCs. The product model keeps the invitation and event operations distinct from public presentation: one physical card belongs to one private invitation, and RSVP, attendance, and claims belong to individual people.

The repository contains an authenticated organizer path, private RSVP path, NFC feasibility code, local-only demo console and guest preview, a GrapesJS Designer Studio, and a proposal-only AI assistant. Batch 5B adds an additive client/event/registration database design and candidate migrations 0007–0009. Those migrations and their RPCs are authored for review only; no database connection or migration was performed. The configured Supabase project is not confirmed as an isolated synthetic target, and its applied migration history is unknown. Public publishing and safe public rendering are not available yet.

```
 Browser clients
    |
    v
Next.js App Router (pages, Server Actions, route handlers, proxy)
    |
    +--> feature modules: auth | organizations | invitations | microsites | NFC | platform
    |
    +--> Supabase Auth + PostgreSQL (RLS policies and transactional RPCs)
    |
    +--> OpenAI Responses API (optional, server-only, validated proposals)

Local demo is a separate development-only path using signed session cookies and browser storage.
```

2. Repository map

Area	Responsibility	Main locations
App Router	Route composition and page-level presentation	`src/app/**`
Request proxy	Supabase session refresh; bypass for enabled local-demo routes	`src/proxy.ts`

Area	Responsibility	Main locations
Authentication	Supabase email/password and MFA; development-only demo credentials and signed cookie	`src/modules/auth/**`
Organizations and events	Workspace/event management, invitation issuance, NFC registry actions and queries	`src/modules/organizations/**`
Invitations	Private-token lookup and per-person RSVP submission	`src/modules/invitations/**`
Microsites	GrapesJS drafts, templates, event access, AI operations/provider and editor UI	`src/modules/microsites/**`
NFC	URL parsing, Web NFC helpers, staff feasibility UI and card writer	`src/modules/nfc/**`
Local platform demo	Synthetic clients/sites, demo dashboard and guest preview	`src/modules/platform/**`
Shared infrastructure	Drizzle schema and Supabase browser/server clients	`src/shared/infrastructure/**`
SQL history	Schema, constraints, functions, RLS, microsite draft/version, and candidate Batch 5B migrations	`drizzle/*.sql`
Database preparation	Migration design, RLS matrix, restore plan, local-only DB integration suite	`docs/DATABASE_BATCH5_DESIGN.md`, `tests/database/`, `tests/fixtures/`
Browser checks	Isolated local-demo Designer Studio workflows	`e2e/`, `playwright.config.ts`

The feature modules expose small public entry points in `public.ts`. In the current implementation, several `use server` action modules still contain input validation, authorization checks, and Supabase RPC calls together. Keep future business rules in named use cases and keep Next.js adapters thin as the architecture evolves.

3. Runtime routes

Route	Role and current boundary
`/sign-in`, `/sign-up`, `/check-email`, `/setup/mfa`	Supabase authentication and MFA setup. When enabled in development, local-demo sign-in is handled separately from Supabase Auth.
`/organizer`	Authenticated organizer workspace and event listing.
`/organizer/events/[eventId]`	Event management, invitations, and NFC card actions. Server actions re-check the signed-in user, verified email, and MFA assurance before manager mutations.
`/organizer/events/[eventId]/designer`	Event-scoped Designer Studio entry for verified/MFA-authenticated event managers. Draft read/write calls depend on SQL RPCs and migration `0006_solid_hydra.sql`.

Route	Role and current boundary
`/invite`	Guest invitation experience driven by a private invitation token. Public lookup and RSVP use constrained SQL RPCs.
`/event/[slug]`	Guest microsite route; local demo resolves synthetic microsites. It does not render an event's current database draft as a published site.
`/admin`, `/admin/designer/[siteId]`, `/demo/microsites/[siteId]`	Local-demo console/editor/preview. These flows are development fixtures with browser-local settings and drafts, not production client data.
`/staff/nfc-prototype`	Staff NFC feasibility interface; physical Android/Chrome and tag behavior still require device validation.
`/api/designer/assistant`	Server-side proposal endpoint. Requires a signed demo session or verified email, MFA AAL2, and scoped event/draft checks for real-event requests.

4. Identity, event and invitation model

The Drizzle model in `src/shared/infrastructure/database/schema.ts` and SQL migrations describe these main relationships:

```
 organizations 1 --- * organization_memberships * --- 1 Supabase Auth user
organizations 1 --- * events 1 --- * invitations 1 --- * attendees
organizations 1 --- * nfc_cards (each active card is bound to one invitation)
events 1 --- 1 microsites 1 --- * microsite_versions
events/organizations --- audit_events and notification_outbox
organizations 1 --- 1 client_profiles
events 1 --- 1 event_settings
events 1 --- * event_registration_configs 1 --- * event_registration_questions
events 1 --- * open_registrations 1 --- * open_registration_people 1 --- * registration_answers
invitations 1 --- * attendees 1 --- * private_registration_answers
open_registrations 1 --- * registration_history
```

`organizations` and `organization_memberships` define tenant ownership and roles.

`client_profiles` adds descriptive client contacts/status on the organization. Contact details never grant permissions; membership remains the authorization source.

`events` carry schedule, timezone, venue, capacity/reserved places, lifecycle status, and the mutually exclusive guest mode (`rsvp`, `meal_booth`, `album`).

`event_settings` stores event category, independent service switches, and bounded event settings without changing the existing lifecycle or guest-mode model.

`event_registration_configs` stores immutable-in-use versions, private/open/approval mode, registration window, approval and waitlist policy. `event_registration_questions` stores stable keys, order, field type, options, validation, and simple conditional visibility.

`open_registrations`, `open_registration_people`, `registration_answers`, and `registration_history` model open submissions and per-person answers/status. Private answers use `private_registration_answers` keyed to existing `attendees` and invitation-pinned form versions.

Open registration and private invitation issuance share the existing `events.reserved_places` counter. Their SQL operations lock the event row so competing capacity changes serialize. A guest cancellation capability uses the high-entropy idempotency key; the database stores its hash.

`invitations` store a token hash, label, primary email, companion allowance, and reserved places. The raw token is returned for the guest link; the database stores its hash.

`attendees` store named people, their per-person attendance status, and check-in timestamp.

`nfc_cards` store card credential hashes, assignment and lifecycle fields. Constraints enforce one active card per invitation and valid replacement/revocation state.

`microsites` hold one event draft as complete GrapesJS JSON and revision metadata. `microsite_versions` is the planned immutable publication-history table.

`audit_events` and `notification_outbox` provide structures for auditable changes and reliable notification delivery; the end-to-end worker/provider path is not complete.

RLS policies and SQL RPCs are the database-side boundary. Batch 5B adds candidate manager-read policies and RPC-only writes for new tables; public registration uses narrowly scoped RPCs. Do not infer database runtime verification from TypeScript or migration source. The local migration journal records 0000–0009, but remote history is unknown. None of Batch 5B was applied and no database runtime verification occurred.

5. Principal request flows

Organizer

The organizer signs in with Supabase Auth, confirms email, and reaches MFA assurance level AAL2.

Organizer pages query only records permitted by RLS.

Server Actions validate form inputs and call SQL RPCs for organization/event, invitation, publication-state, and card operations.

The RPC and RLS layer enforces event/tenant permissions and relevant integrity constraints.

Guest RSVP

A private invitation link carries a random token; the database retains only the token hash.

A public lookup RPC resolves safe event/invitation fields and current guest mode.

The guest submits the primary email and named per-person attendance responses.

The RSVP RPC performs server-side validity, deadline, status, duplicate-submission, and allowance checks. The UI reports success only after the RPC confirms the write.

Open registration and RSVP operations

A public form RPC returns public event metadata and fields from the current versioned configuration; direct anonymous table reads remain disabled.

The submit RPC hashes a high-entropy idempotency key and request payload, locks the event, validates per-person answers, and checks the shared capacity counter.

It writes registration, people, answers, reservation, history, audit, and notification-outbox records atomically. Repeated key/payload returns the existing result; key reuse with a changed payload is rejected.

Managers with event authorization and MFA can approve, reject, cancel, or promote a waitlisted registration. Guests can cancel using the event-scoped idempotency capability. These flows are authored but not database-tested or connected to application UI.

Designer draft

The organizer opens Designer Studio for an existing event after verified email, MFA, and event authorization.

GrapesJS runs client-side and stores a complete project document, including pages, components, styles, and metadata.

The draft action applies a 2 MB bound, JSON-content checks, authorization and expected-revision checks before calling the save RPC.

A stale revision is rejected to prevent overwriting another edit session. The local-demo path uses browser `localStorage` and is explicitly separate from event persistence.

Draft preview as a guest site, publishing, immutable snapshots, restore/history, and safe public rendering remain future work.

AI design proposal

The browser sends a bounded prompt, recent conversation, and compact structure: page IDs/names/slugs plus component IDs/types/tags. It omits component copy, styles, guest lists, RSVP details, seating, and credentials.

The route authorizes the caller and validates request size/context. The provider is either an explicit deterministic development demo or server-side OpenAI Responses API.

Zod constrains output to a maximum of eight whitelisted design operations. Client-side context checks run again.

The UI previews effects and destructive changes; the user must explicitly apply. A pre-apply project snapshot supports undo.

The provider cannot return executable HTML/JavaScript, arbitrary CSS, commands, or raw project replacement. Codex execution is not integrated into the product.

6. Trust boundaries and safety properties

Credentials: Root `.env` is the local default via `dotenv/config` and remains ignored. `.env.example` contains placeholders only. Supabase URL/publishable-key values are public client configuration; service-role or other privileged values stay server-only. Never commit or print local credentials.

Demo login: only enabled under development and an explicit flag; a signed HttpOnly, SameSite cookie identifies the local fixture. The local demo does not use the production user/tenant database.

Organizer actions: validate server-side identity, verified email, MFA level, tenant/event resource access, and payload before mutation.

Guest data: keep invitation access, named attendees, event staff sessions, and platform support grants separately scoped. Never place private guest information in a public designer project.

NFC: a scan identifies or validates; a staff confirmation is the mutation boundary. QR and authorized manual lookup remain fallbacks. No offline support or clone-proof claim is established.

Designer content: uploads/import and published rendering are unavailable. The draft validator is a guardrail, not a publication sanitizer; a future public renderer must use a separate safe allowlist.

AI: the API key is server-side, requests use `store: false`, output is typed and bounded, and all changes require explicit review/apply.

Database: configured project ref `zktyesiawrwfybcllnkr` and database host are not independently confirmed as isolated synthetic development. Do not connect or apply migrations until target identity and data isolation are established, remote migration history is inspected, and explicit approval is given. No Batch 5B SQL was applied.

Open guest cancellation: the idempotency key is a bearer capability. Client code must generate it with a cryptographically secure random source, retain it for the guest, and never log or expose it. Application-level rate limiting/CAPTCHA and key recovery remain outstanding.

7. Local development and verification

Use pnpm and the ignored `.env` for local configuration:

```
 pnpm install
pnpm dev
pnpm typecheck
pnpm exec biome check src/modules/microsites/ui/designer-studio.tsx src/modules/microsites/ai
src/app/api/designer/assistant/route.ts e2e playwright.config.ts
pnpm exec drizzle-kit check
pnpm test:db:local
pnpm build
pnpm test:e2e
```

`pnpm test:db:local` requires explicit `MOMENTUS_TEST_DATABASE=1` and a loopback-only `DATABASE_URL` in root `.env`; otherwise it skips database connections. Use only a disposable local database with the reviewed migrations already applied.

Batch 5B checks: `pnpm typecheck`, targeted Biome, `pnpm exec drizzle-kit check`, and schema-diff generation pass. `pnpm test:db:local` safely skipped its seven database integration cases because `.env` identifies a non-loopback Supabase host; no database connection was attempted. Supabase CLI, `psql`, and Docker are unavailable. Earlier local-demo Playwright results are recorded in the acceptance log; they were not rerun for this database-preparation update. No PostgreSQL parser/runtime check or Next.js build was run for the candidate SQL.

8. Delivery status and next seams

Area	Current state	What evidence or implementation remains
Supabase Auth / MFA	Implemented in source	Real account setup, redirect allowlist and full auth acceptance remain to verify.
Organizer / events / invitation issue	Implemented in source with RLS/RPC calls	Confirm isolated development schema and exercise real database workflows.
Client/event registration schema	Candidate migrations 0007–0009, backfill, RLS and transactional RPCs authored	Confirm target/history, review SQL and backup, obtain approval, then run integration/RLS/concurrency tests. Application actions/UI are not connected to new RPCs.
Public RSVP	Existing private invitation flow plus additive private answer-aware and open registration RPC designs	Database workflow/concurrency checks, application integration, rate limiting and outbound email remain.
NFC	URL helpers, card writer/registry code and staff feasibility prototype	Physical Android Chrome/tag test, staff confirmation workflow and production operational evidence remain.
Designer Studio	GrapesJS Core, 12 templates, Batch 1 controls and Batch 2 pages/RTE/navigation/productivity/assistant	Real database draft roundtrip, shortcut and broader RTE tests, media/import, publication snapshots, restore, and safe renderer remain.
AI assistant	Server-only OpenAI adapter and deterministic local provider	No live provider/account test; keep disabled until key, cost, and authorization are intentionally configured.
Email, claims, check-in, memories	Data model and some guest-mode structures exist	Delivery worker, operational mutation services, real guest pages, album/media lifecycle and supervised flow are outstanding.
Deployment / recovery	Local-first development; intended self-hosted production stack	Supabase CLI, `psql`, and Docker are unavailable. Production platform/recovery is deferred; physical validation, provider setup and pilot gates remain open.

The next database step is to establish a disposable Supabase-compatible local target, inspect migration prerequisites/history, and obtain explicit approval before applying migrations. Then run the prepared tenant-isolation, form-persistence, capacity, retry, cancellation, and transition cases. Separately, connect the new RPCs to server actions and UI. Designer draft database validation, immutable publish snapshots, and an allowlisted public renderer remain independent follow-on work.

9. Source map

Product behavior/approved scope: `docs/DECISIONS.md`, `docs/IMPLEMENTATION_PLAN.md`, `docs/ACCEPTANCE.md`

Authentication and request proxy: `src/modules/auth/`, `src/proxy.ts`

Organization/event actions and queries: `src/modules/organizations/`

Private invite/RSVP: `src/modules/invitations/`

Designer, drafts, templates and assistant: `src/modules/microsites/`

Database schema and reviewed SQL: `src/shared/infrastructure/database/schema.ts`, `drizzle/`

Batch 5B database design/approval gates: `docs/DATABASE_BATCH5_DESIGN.md`

Synthetic fixtures and guarded DB integration suite: `tests/fixtures/batch5-synthetic.json`, `tests/database/batch5.integration.test.mjs`

Public route entry points: `src/app/`

Local browser test configuration: `playwright.config.ts`, `e2e/designer-studio.spec.ts`